

Mechanism for Long Context for text-based LLMs

Aditya Vishwakarma

Abstract

*The paper describes a novel **Divided Context Architecture (DCA)** for Large Language Models (LLMs) that aims to increase context length without proportionally increasing computational demands or memory usage. The core innovation lies in modifying the **attention mechanism** through the introduction of **vertical retention vectors** and a new **LOTA (Least of Them All) normalisation function**, which performs global score normalisation. The architecture is composed of stacked "**Blocks**," with each block containing "Parallel" and "Partial Attention" units that share information via these retention vectors, facilitating both horizontal and vertical information flow.*

Introduction

Based on the transformer introduced by Vaswani et. Al. DCA introduces new weights and vectors for retaining and increasing context. This is very specific modification to transformer structure where multiple such transformers are stacked over each other and the information is shared to next transformer via vertical retention vectors. The major change is in attention mechanism with new mlp for vertical retention vectors and also vertical weight matrices.

For feed forward, there are two MLPs, first one for horizontal and second one is vertical MLP, for horizontal and vertical retention vectors, where horizontal vertical vector E_H which is single vector with dimension equal to dimension of tokens embeddings, while E_V is matrix with size as context-window times dimension of token embeddings. For attention score normalisation, the new function is LOTa, an acronym for Least of Them ALL. This is done for global score normalisation, rather than row-wise SoftMax. In this function, the minimum value from dot product grid is obtained, and then its absolute value is added to each and every value of grid, then the sum of new values is obtained. After this, the scores are normalised by dividing each value by this sum. Unlike SoftMax where masking is done in self-attention, here it does not need to be done.

Structure

A matrix of attention mechanism is called as Block, with columns called as PARALLEL and rows are PARTIAL ATTENTION (PA). Multiple blocks are called as transformer. Context window of single block is called as LOCAL CONTEXT (C_L). The

whole transformer has FULL CONTEXT (C_F). The local context is equal for all blocks in DCA. The Dimensions of components are as follows:

- Context Window: C_W , Local Context = C_L , Full Context: C_F
- Number of Blocks: N_B , Number of PARALLELS: N_P , Number of PAs: N_{PA}
- Number of Attention units per PA = N_A
- Dimension of EMBEDDING: D_E
- Weight Matrix Heights: D_H
- M_Q, M_K : $D_E \times D_H$
- M_H, M_V : $D_H \times D_E$
- QK', KH, QV : $D_E \times D_E$
- $KdotQ$: $C_L \times C_L$
- Dimension of Input and output vectors of MLP: D_E
- MLP weights matrix: $D_E \times D_E$
- Number of Weights Matrix of MLP: n_L
- FULL CONTEXT: $C_F = N_B \times C_L$

Attention Mechanism

Attention mechanism is the unit process of every transformer, with weights and mlp and respective weight matrices. Here are the components of DCA attention mechanism:

- Q: Query vectors
- K: Key vectors
- Query weights: M_Q
- Key Weights: M_K
- Horizontal Weights: M_H
- Vertical Weights: M_V
- Hor and Ver mlp
- Compressed Weights Caches: QK', QV, KH
- $KdotQ$: Query and Key dot product grid (without normalisation)
- dh and dv : scaled dot product for horizontal and vertical retention vectors
- EH : horizontal retention vector
- EV : vertical retention vectors (matrix)

The dot products are divided by square root of dimensions of embedding given by Vaswani et. Al., which is followed in DCA also. $KdotQ$ can simply be put as product of Q and Transpose of K .

$$KdotQ = \frac{Q \times K^T}{\sqrt{D_E}}$$

$$Q(i) = T(i) \times M_Q, \quad K(i) = T(i) \times M_K$$

$T(i)$ = i^{th} token embedding.

$$\begin{aligned} KdotQ(i, j) &= \frac{Q(i) \cdot K(j)}{\sqrt{D_E}} \\ &= \frac{Q(i) \times K(j)^T}{\sqrt{D_E}} \\ &= \frac{[T(i) \times M_Q] \times [T(j) \times M_K]^T}{\sqrt{D_E}} \end{aligned}$$

Since, $(A \times B)^T = B^T \times A^T$

$$\begin{aligned} KdotQ(i, j) &= \frac{[T(i) \times M_Q] \times [M_K^T \times T(j)^T]}{\sqrt{D_E}} \\ &= \frac{T(i) \times (M_Q \times M_K^T) \times T(j)^T}{\sqrt{D_E}} \\ \mathbf{QK^T} &= \mathbf{M_Q} \times \mathbf{M_K^T} \end{aligned}$$

For inference, it becomes:

$$KdotQ(i, j) = \frac{T(i) \times \mathbf{QK^T} \times T(j)^T}{\sqrt{D_E}}$$

LOTA

Least of Them All is the method to normalise the values of set via most minimum value from the set and then calculate sum of these new values and divide each of them by this sum. Consider $KdotQ$ to be a matrix of values, $KdotQ_{\min}$ is the minimum value out of all. Each value of the grid is subtracted by this minimum value and the sum s is calculated from these new values, which are all positive. These positive values are fraction of this sum s and hence when divided by s , what we get is globally normalised score, fractions of these values w.r.t s .

$$LOTA(i, j) = \frac{KdotQ(i, j) - KdotQ_{\min}}{s}$$

$$s = \sum_i^{limit} (\sum_j^{limit} (KdotQ(i, j) - KdotQ_{\min}))$$

Now, for upgrading vectors dh and dv via normalised scores for Keys and Queries:

$$\begin{aligned}
dh &= \sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(i, j) \right) \cdot K(i) \times M_H \\
&= \left(\sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(i, j) \right) \cdot T(i) \right) \times M_K \times M_H \\
dv &= \sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(j, i) \right) \cdot Q(i) \times M_v \\
&= \left(\sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(j, i) \right) \cdot T(i) \right) \times M_Q \times M_V
\end{aligned}$$

For inference, they take the form of:

$$\begin{aligned}
\mathbf{M}_K \times \mathbf{M}_H &= \mathbf{KH}, \quad \mathbf{M}_Q \times \mathbf{M}_V = \mathbf{QV} \\
dh &= \left(\sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(i, j) \right) \cdot T(i) \right) \times \mathbf{KH} \\
dv &= \left(\sum_i \left(\sum_j^{(isSelf)? i : limit} LOTA(j, i) \right) \cdot T(i) \right) \times \mathbf{QV}
\end{aligned}$$

This modification done here provides two types of weights matrices, first one is trainable and second one is compressed weights. The four trainable weights matrices are $\mathbf{M}_Q$, $\mathbf{M}_K$, $\mathbf{M}_H$ and $\mathbf{M}_V$ and compressed weights are referred as caches, $\mathbf{QK}^T$, $\mathbf{KH}$ and $\mathbf{QV}$ respectively. These compressed weights are used for inference.

Limit is used to describe current token count, which includes, total tokens of prompts and previous response and also current response. The two activation functions used are ReLU and Sigmoid, where Sigmoid is used in mlp activations, and ReLU is used for activations of Logits (mostly mlp outputs).

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}, \quad \sigma(x) = \frac{1}{1 + e^{-x}}$$

For retention vectors:

$$hor.input = EH + dh$$

$$For\ each\ i\ in\ 1\ to\ local\ context: ver.input += EV(i)$$

$$ver.input += dv$$

$$EH = EH + \text{ReLU}(hor.output)$$

$$\text{For each } i \text{ in } 1 \text{ to local context: } EV(i) = EV(i) + \text{ReLU}(ver.output)$$

Block

A single block is similar to a transformer (Vaswani et. Al.), a 2-dimensional arrangement of attention units, with each unit sharing EH to next one in same PA and EVs to same subsequent unit of next block. The forward propagation and backward propagation i.e., Forprop and Backprop, are executed in parallel through column-based execution. Block also has a common collection of all the EVs from each attention unit and a separate token embedding matrix to hold tokens specific to each block. Columns are referred as **PARALLEL** since they are executed in parallel, while the end head's EH is used for final EH vector (which is either sum of concatenation of all end head's EH of each row), hence they are called **PARTIAL ATTENTION**. Hence the block can be referred as **COMPLETE ATTENTION**.

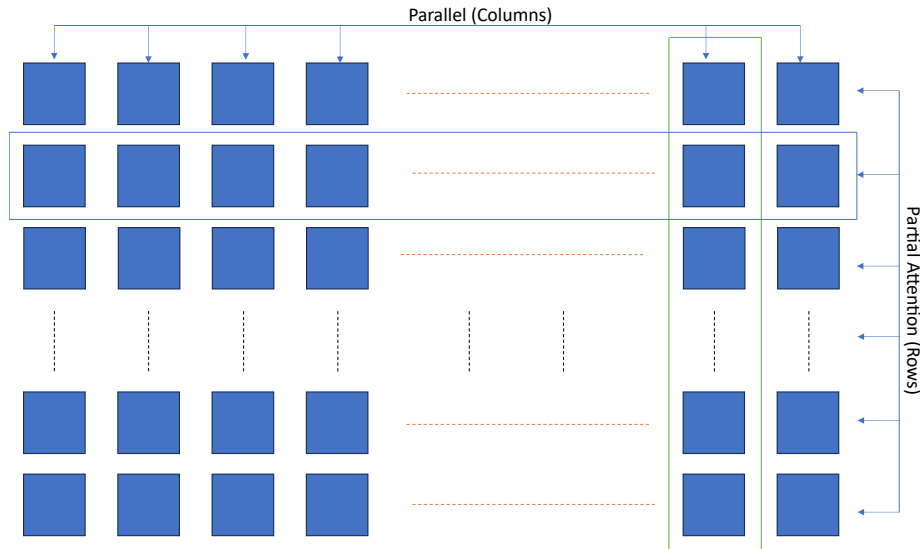


Fig.: Single Block

Transformer

Transformer in DCA is stacked arrangement of Blocks. Here all processes run block-wise. In training, Forprop works on single block, but for backprop, starting from the current block up to first block, process is done. This to correct the vertical retention as much as possible. In inference, only one block is defined, and used also, when context for block is reached, the EVs of same block are used for queries and embeddings from previous context for keys. In inference, Caches

are used rather than trained weights. These caches are obtained by multiplying trained weight matrices. Since, the total context of transformer is Local context times number of blocks, transformer is also referred as **FULL CONTEXT**.

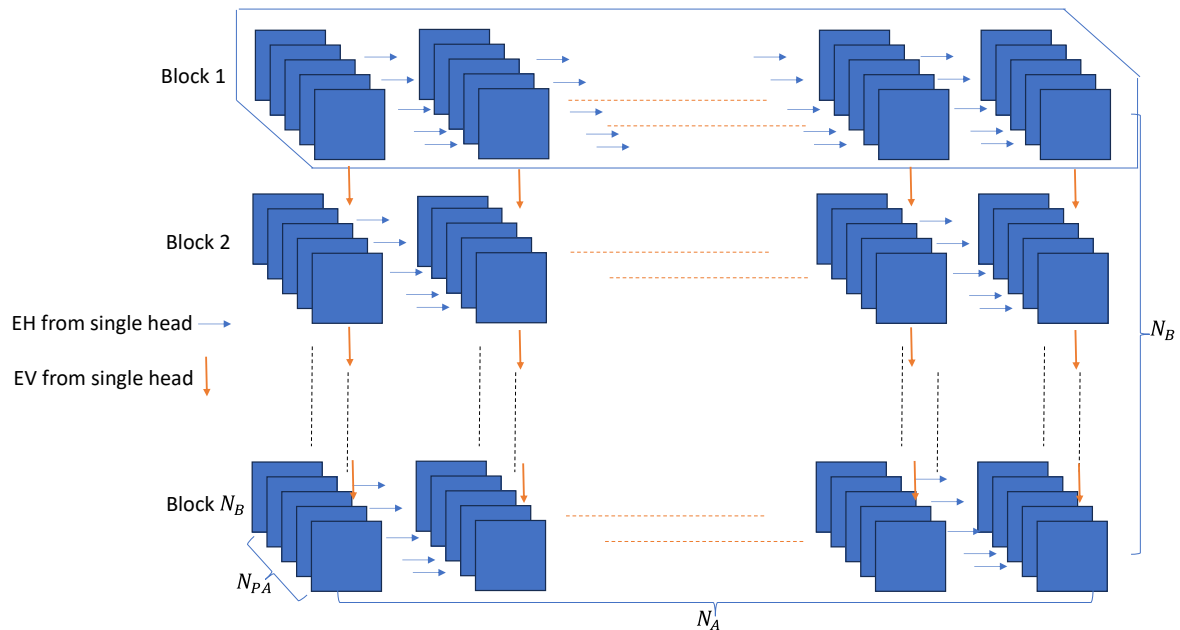


Fig.: DCA Structure (Transformer)

Forprop

For first block, the Forprop uses embeddings, for other blocks, it uses EVs from previous blocks and token embeddings used in previous blocks.

As provided for attention mechanism, the Forprop process is similar for first and other blocks. It starts from embeddings (and EVs in next block), then Queries and Keys are obtained, dot product is calculated, current tokens, and then scaled dot product vectors are calculated, for both horizontal retention for token prediction and vertical retention for context retention in next block. Then inputs are provided to MLPs, and the output of MLPs are logits which are ReLUed, and then used to update the EH and EV. Updated EH is passed to next unit, and this update continues till last unit is reached. As Local context is reached, EV keeps on updating.

Once Local Context is reached, the process shifts to next block and uses EVs from previous block to calculate queries. Tokens of previous Local context are used for key calculations.

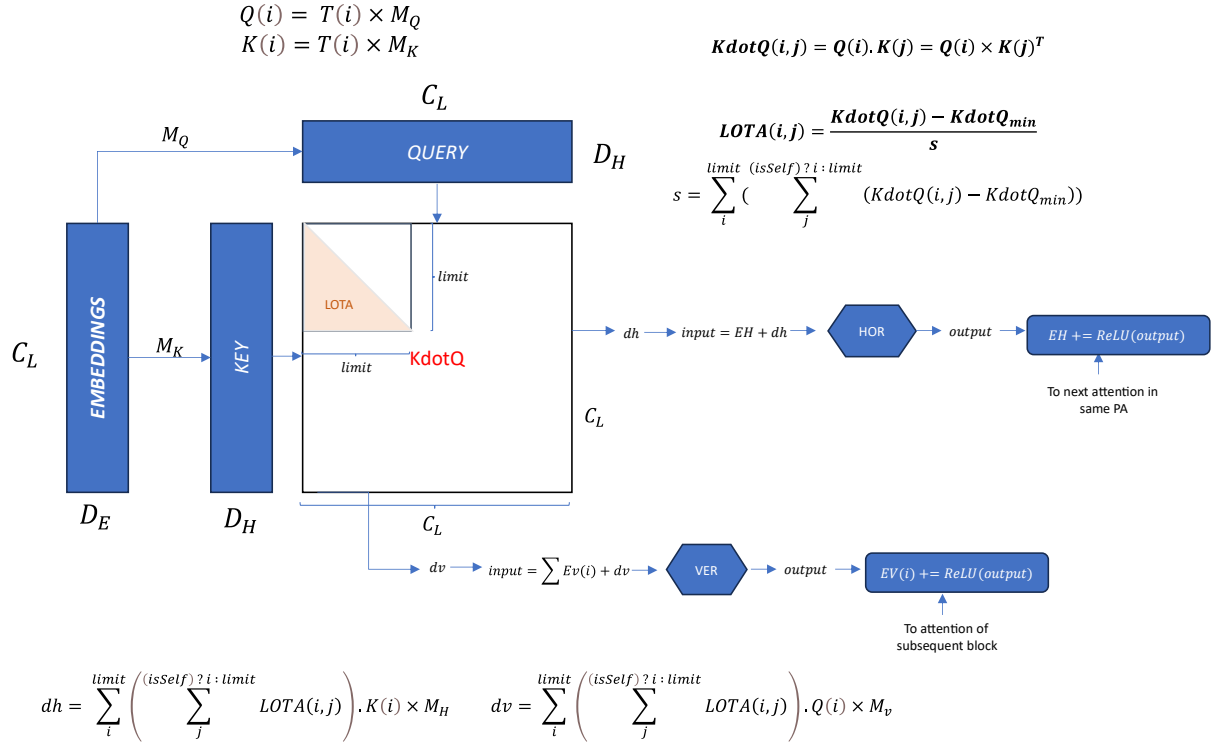


Fig.: For attention units in first block

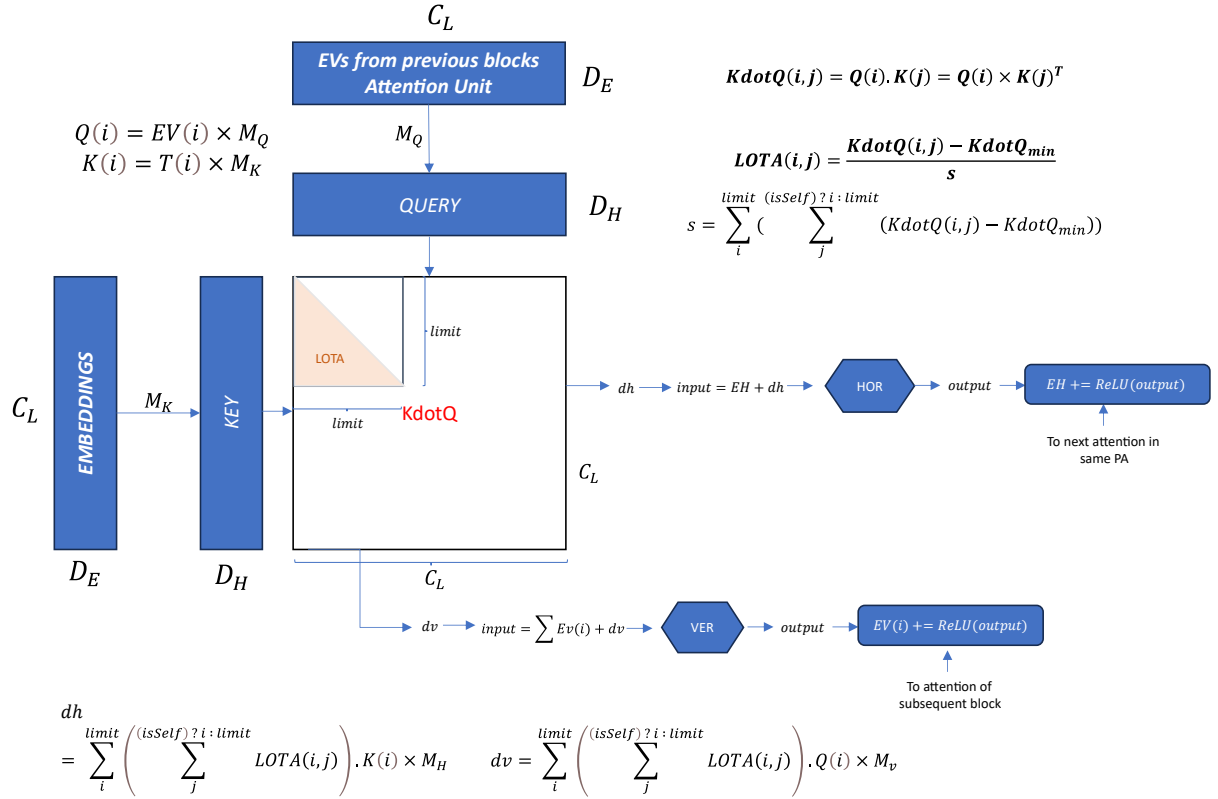
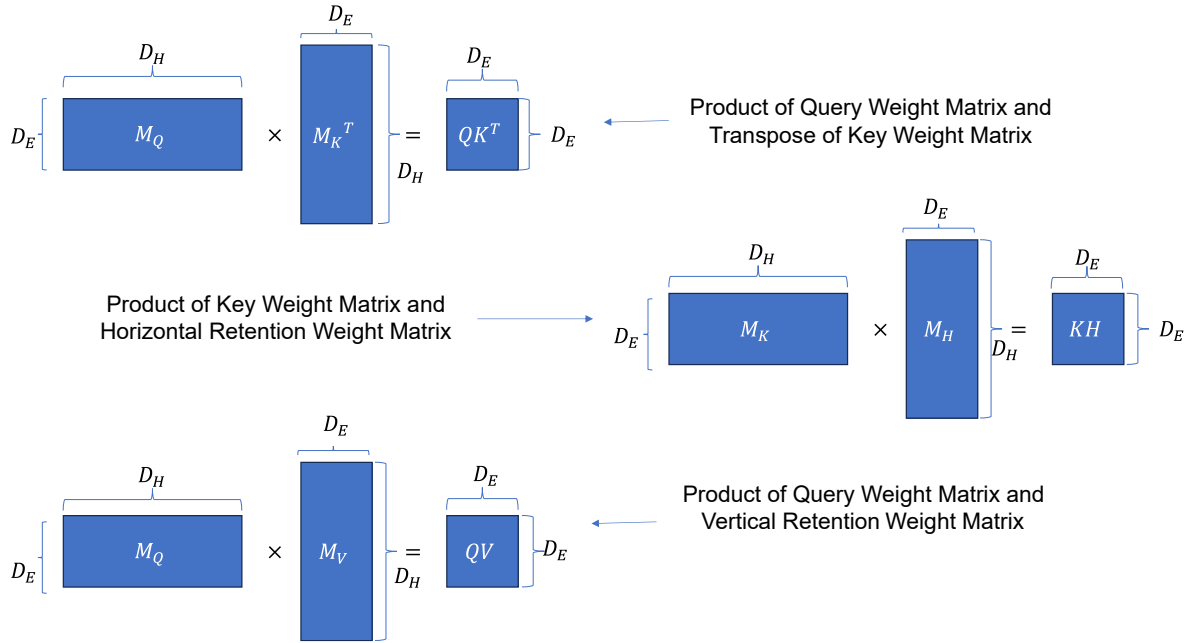


Fig.: For attention units of other blocks

Compared to trained matrices, caches can be calculated after training is done. Caches reduce parameters and compute. While trainable matrices can hold more parameters and need more compute, in training, they can hold more nuances and features. This allows, to train model on larger data, extract more features and in inference, use the low parameter caches without losing features.



Backprop

Suppose i^{th} block is the current block, where the process is taking place. The backprop starts from expected token, from last column to first, and this block to first block. In the same block, backprop works on the basis of EH and for other upward blocks, $(i - 1)^{\text{th}}$ to 1^{st} , based on the EVs of the i^{th} to 2^{nd} block. Hence, there are two types of backpropagations. This can be further be classified for four cases based on how EH and EVs are updated based on the head location and block.

Following are ReLU, Sigmoid and LOTA derivatives, which are necessary for calculating gradients.

$$ReLU'(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

$$LOTA'(i, j) = \frac{1 - LOTA(i, j)}{s}$$

Backprop actually starts due to wrong token prediction and training and then expected token is used to calculate correct tokens. This process starts from last column till first column, and at every head of column, corrects weights matrices and mlp weight layers, using gradients. For this it utilises a cost function, for gradient calculation of EH/expected token, then the gradients are used to calculate an expected vector for both hor and ver mlp. Since the token prediction is dependent on EH, EV does not get an expected vector, hence its gradient is considered to be a fraction of gradient of EH. This can be less than or equal to Learning rate.



$$gEV(i) = f * gEH(i), \quad f \leq \text{learning rate } (R_L)$$

The first type is **Backpropagation via Expected Tokens**, which is primarily triggered by a **wrong token prediction**. Its aim is to adjust the model's parameters to correct the predicted token towards the Expected token. In this process, the gradient for the horizontal retention vector EH (g_{EH}) is calculated directly from the difference between the current EH and the Expected token, following a Mean Squared Error (MSE) like formula: $g_{EH}(i) = 2 * (EH(i) - Expected(i))$. Crucially, the gradient for the vertical retention vector EV (g_{EV}) is derived as a **fraction of $g_{EH}(i)$** , where f is less than or equal to the LEARNING rate (R_L), ($g_{EV}(i) = f * g_{EH}(i)$). This design choice indicates that updates to EV stemming from token prediction errors are proportionally smaller, likely to help preserve the stability of context information in EV. This backpropagation pathway results in updates to **all four trainable weight matrices: MQ, MK, MV, and MH**, as well as both EH and EV. The corrections propagate backward from the last column to the first within a block, and upward from the current block to the first block.

Backprop Via Expected Vertical Retention Vector

When backprop, is calculated for i th block for expected token, it actually updates its EVs, which are taken as expected EVs for $(i-1)$ block, going upward. A collective gradient is calculated for this process.

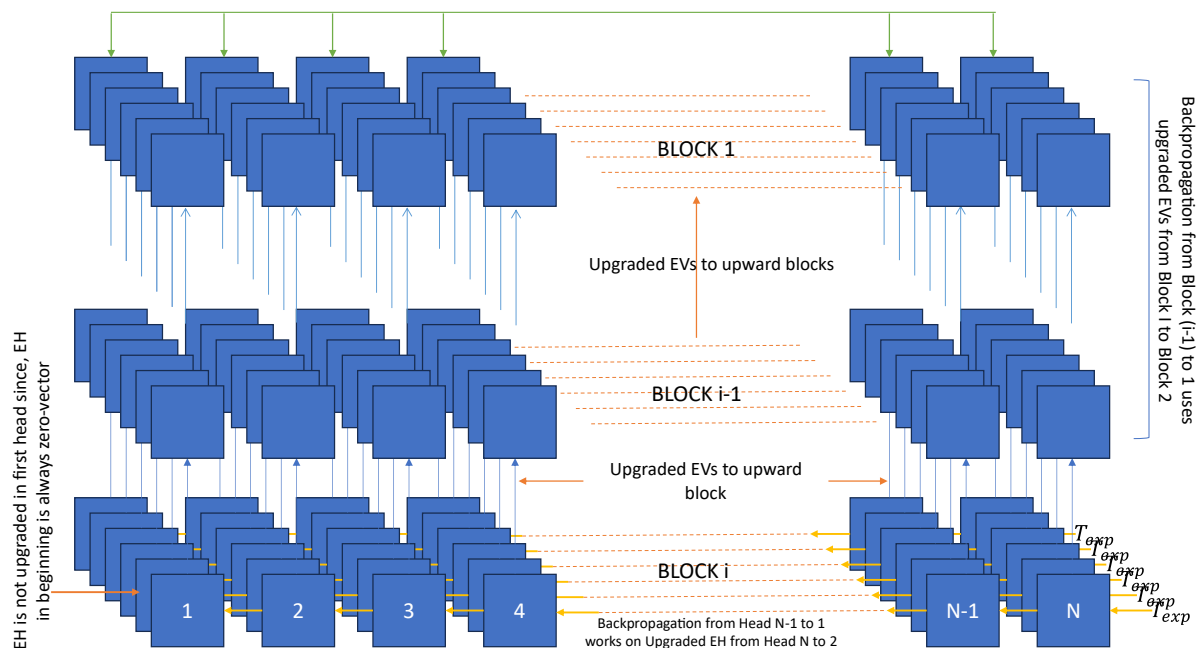


Fig.: Backprop from i^{th} Block to 1^{st} Block for Token Prediction

The second type is **Backpropagation via Expected Vertical Retention Vector**, which focuses specifically on **correcting context retention** errors. This occurs when the EVs from a subsequent block are used as the expected EVs for a preceding block, propagating corrections "upward" through the stack of blocks. The objective here is to **adjust MQ and MV** to ensure context integrity, while aiming to keep the horizontal pass matrices (MK and MH) largely unaffected by these specific context-focused corrections. In this scenario, the gradient for EV (grad_EV_mat) is calculated element-wise based on the difference between $\text{EV}(j, i)$ and $\text{expectedV}[j][i]$ ($2.0 * (\text{EV}(j, i) - \text{expectedV}[j][i])$). This element-wise gradient is then summed and averaged (grad_EV_summed) to provide input for the vertical MLP's backward pass¹¹. Unlike the EH-based backpropagation, the MH matrix is **not updated** in this path, and MK receives a specific $\text{grad_MK_correction}$ rather than a general update. Therefore, this pathway primarily updates **MV and MQ**, with a calculated adjustment to MK, and updates the EV matrix using its direct element-wise gradients.

Reference

1. Vaswani, Ashish, Shazeer, Noam, Parmar, Niki, Uszkoreit, Jakob, Jones, Llion, Gomez, Aidan N., Kaiser, Lukasz, and Polosukhin, Illia. Attention is all you need. 2017. <https://doi.org/10.48550/arXiv.1706.03762>